

Optimization Methods for Machine Learning and Engineering

Lecture 2 – Newton Method, Loss Functions and Regularization

Julius Pfrommer



CC BY-SA 4.0

1. Linear Algebra Recap

2. Newton's Method

3. Model Fitting as Optimization

Maximum Likelihood Estimation

Regression

Loss Functions

Regularization

Linear Algebra Recap

The Fibonacci Series

1, 1, 2, 3, 5, 8, 13, 21, 34, ...

The **Fibonacci Series** is defined by a recursive rule:

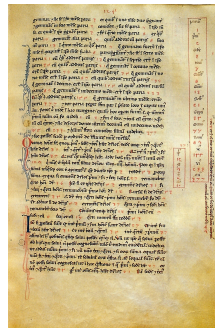
$$n_1 = 1$$

$$n_2 = 1$$

$$n_{k+2} = n_k + n_{k+1}$$

- **Occurs in nature** (Sunflower seed pattern, no. of male ancestors in the family tree, etc.)
- Important for the **runtime analysis** of certain algorithms (e.g. Merge-Sort)
- Studied in early Sanskrit poetry (ca. 450BC)

Liber Abaci (1202) with the Fibonacci Series



How fast can Fibonacci numbers be computed?

Recursive

```
function fib(k)
    if k <= 2
        return 1
    end
    return fib(k-2) + fib(k-1)
end
```

Exponential runtime of the recursive algorithm!

```
@time fib(10) # 0.000002 seconds
@time fib(30) # 0.003058 seconds
@time fib(49) # 27.321591 seconds
@time fib(50) # 44.748634 seconds
```

Memoization

```
function fib(k)
    res = zeros(k)
    res[1] = 1
    res[2] = 1
    for i in 3:k
        res[i] = res[i-2] + res[i-1]
    end
    return res[k]
end
```

Linear runtime with memoization

```
@time fib(10)      # 0.000003 seconds
@time fib(100)     # 0.000004 seconds
@time fib(1000)    # 0.000007 seconds
@time fib(10000)   # 0.000055 seconds
```

Vector Multiplication

Column Vector

$$\mathbf{v} = \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix}$$

Row Vector / Transpose

$$\begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix}^T = (v_1 \quad v_2 \quad v_3)$$

Vector Multiplication

$$\mathbf{x}^T \mathbf{y} = (x_1 \quad x_2) \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = x_1 y_1 + x_2 y_2$$

Unless indicated otherwise in the notation, **all vectors are column-vectors** in this course.

```
v = [1;2;3]
# 3-element Array{Int64,1}:
#  1
#  2
#  3
```

```
u = [7 8 9]
# 1×3 Array{Int64,2}:
#  7  8  9
```

```
v'
```

```
# 1×3 Array{Int64,2}:
#  1  2  3
```

```
u * v
# 50
```

```
v' * v
# 14
```

Matrix-Vector Multiplication

Matrix and Column Vector

$$\mathbf{A}\mathbf{v} = \begin{pmatrix} \text{---} & \mathbf{a}_1^\top & \text{---} \\ \text{---} & \mathbf{a}_2^\top & \text{---} \\ \text{---} & \mathbf{a}_3^\top & \text{---} \end{pmatrix} \begin{pmatrix} | \\ \mathbf{v} \\ | \end{pmatrix} \\ = \begin{pmatrix} \mathbf{a}_1^\top \mathbf{v} \\ \mathbf{a}_2^\top \mathbf{v} \\ \mathbf{a}_3^\top \mathbf{v} \end{pmatrix}$$

```
A = [1 2 3;  
      4 5 6]
```

```
v = [1;2;3]
```

```
A * v
```

```
# 2-element Array{Int64,1}:
```

```
# 14
```

```
# 32
```

Row Vector and Matrix

$$\mathbf{v}^\top \mathbf{A} = \begin{pmatrix} \text{---} & \mathbf{v}^\top & \text{---} \end{pmatrix} \begin{pmatrix} | & | & | \\ \mathbf{a}_1 & \mathbf{a}_2 & \mathbf{a}_3 \\ | & | & | \end{pmatrix} \\ = \begin{pmatrix} \mathbf{v}^\top \mathbf{a}_1 & \mathbf{v}^\top \mathbf{a}_2 & \mathbf{v}^\top \mathbf{a}_3 \end{pmatrix}$$

```
A = [1 2 3;  
      4 5 6]
```

```
v = [1;2]
```

```
v' * A
```

```
# 1x4 Array{Int64,2}:
```

```
# 9 12 15
```

Matrix Multiplication

$$\begin{aligned} \mathbf{AB} &= \begin{pmatrix} \text{---} & \textcolor{red}{a}_1^\top & \text{---} \\ \text{---} & \textcolor{blue}{a}_2^\top & \text{---} \end{pmatrix} \begin{pmatrix} \left| \right. & \left| \right. & \left| \right. \\ \textcolor{green}{b}_1 & \textcolor{green}{b}_2 & \textcolor{green}{b}_3 \\ \left| \right. & \left| \right. & \left| \right. \end{pmatrix} \\ &= \begin{pmatrix} \textcolor{red}{a}_1^\top \textcolor{green}{b}_1 & \textcolor{red}{a}_1^\top \textcolor{green}{b}_2 & \textcolor{red}{a}_1^\top \textcolor{green}{b}_3 \\ \textcolor{blue}{a}_2^\top \textcolor{green}{b}_1 & \textcolor{blue}{a}_2^\top \textcolor{green}{b}_2 & \textcolor{blue}{a}_2^\top \textcolor{green}{b}_3 \end{pmatrix} \end{aligned}$$

- The matrix dimensions have to match: $\cdot : \mathbb{R}^{n \times \textcolor{red}{k}} \times \mathbb{R}^{\textcolor{red}{k} \times m} \rightarrow \mathbb{R}^{n \times m}$
- The algorithmic runtime is $O(n k m)$
- The transpose of multiplication reverses the order

$$(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$$

- If the matrix has an inverse (for that it must be square), then the inverse commutes

Rules for Matrix Multiplication

Matrix Multiplication is **Associative**

$$[AB] C = A [BC]$$

$$\left[\begin{pmatrix} 1 & 2 & 1 \\ 2 & 0 & 1 \end{pmatrix} \begin{pmatrix} 3 & 2 \\ 1 & 1 \\ 0 & 1 \end{pmatrix} \right] \begin{pmatrix} 2 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 15 & 5 \\ 17 & 6 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 2 & 1 \\ 2 & 0 & 1 \end{pmatrix} \left[\begin{pmatrix} 3 & 2 \\ 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 1 & 0 \end{pmatrix} \right] = \begin{pmatrix} 15 & 5 \\ 17 & 6 \end{pmatrix}$$

Matrix Multiplication is **Not Commutative**

$$AB \neq BA$$

$$\begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & 2 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 4 \\ 1 & 1 \end{pmatrix}$$

$$\begin{pmatrix} 0 & 2 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 2 \\ 1 & 3 \end{pmatrix}$$

Fibonacci even faster with Linear Algebra

Write Fibonacci-steps as a **matrix-vector multiplication**:

$$\begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$
$$\begin{pmatrix} 3 \\ 2 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \underbrace{\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix}}_{= \begin{pmatrix} 2 \\ 1 \end{pmatrix}}$$

$$\begin{pmatrix} n_{k+2} \\ n_{k+1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^k \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

The matrix-exponentiation can be done in ca. $O(\log k)$ steps.

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^8 = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{2^{2^2}}$$

```
function fib(k)
    A = [1 1; 1 0]
    b = [1;1]
    res = A^(k-2) * b
    return res[1]
end
```

Logarithmic runtime as the exponentiation can be computed efficiently.

```
@time fib(100)    # 0.000002 seconds
@time fib(1000)   # 0.000003 seconds
```

But we can do even better!

Matrix Inversion

We have seen matrix powers $\mathbf{A}^k$. Can k be negative?

Yes! Some (but not all matrices) have an **inverse** $\mathbf{A}^{-1}$:

$$\mathbf{A}\mathbf{A}^{-1} = \begin{pmatrix} 1 & 0 & 0 & \cdots \\ 0 & 1 & 0 & \cdots \\ 0 & 0 & 1 & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix} = \text{diag}(1) = \mathbf{I}$$

The **Identity Matrix** “does nothing”: $\mathbf{A}\mathbf{I} = \mathbf{I}\mathbf{A} = \mathbf{A}$

A matrix **does commute with its inverse**: $\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$

The matrix inverse can be computed via **Gaussian Elimination** in $O(n^3)$ for $\mathbf{A} \in \mathbb{R}^{n \times n}$. The fastest algorithms have a runtime of ca. $O(n^{2.37})$. This is still **prohibitive for large matrices!**

```
A = [1 2; 3 4]
```

```
inv(A)
```

```
# 2x2 Array{Float64,2}:
```

```
#  -2.0    1.0
```

```
#   1.5   -0.5
```

```
A * inv(A)
```

```
# 2x2 Array{Float64,2}:
```

```
#  1.0    0.0
```

```
#  0.0    1.0
```

Fibonacci as a closed-form expression!

If a matrix $\mathbf{A}$ is symmetric (sufficient, but not necessary), then it can be factorized with an **Eigendecomposition** as $\mathbf{A} = \mathbf{C}\mathbf{\Lambda}\mathbf{C}^{-1}$ for some $\mathbf{C} \in \mathbb{R}^{n \times n}$ and $\mathbf{\Lambda} = \text{diag}(\boldsymbol{\lambda})$ with $\boldsymbol{\lambda} \in \mathbb{R}^n$.

For the Fibonacci example $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$ we have

$$\mathbf{C} = \begin{pmatrix} \frac{1+\sqrt{5}}{2} & \frac{1-\sqrt{5}}{2} \\ 1 & 1 \end{pmatrix}, \quad \mathbf{\Lambda} = \begin{pmatrix} \frac{1+\sqrt{5}}{2} & 0 \\ 0 & \frac{1-\sqrt{5}}{2} \end{pmatrix}.$$

Taking the k th power of $\mathbf{A}$ simplifies:

$$\begin{aligned} \mathbf{A}^k &= (\mathbf{C}\mathbf{\Lambda}\mathbf{C}^{-1})^k \\ &= \mathbf{C}\mathbf{\Lambda} \underbrace{\mathbf{C}^{-1} \cdot \mathbf{C}}_{\mathbf{I}} \mathbf{\Lambda} \underbrace{\mathbf{C}^{-1} \cdot \mathbf{C}}_{\mathbf{I}} \mathbf{\Lambda} \mathbf{C}^{-1} \dots \\ &= \mathbf{C} \text{diag}(\boldsymbol{\lambda})^k \mathbf{C}^{-1} \end{aligned}$$

```
using LinearAlgebra
```

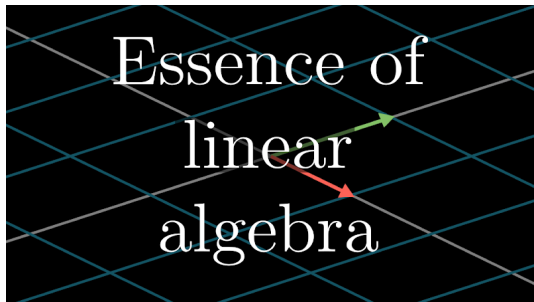
```
function fib(k)
    A = [1.0 1.0;
         1.0 0.0]
    b = [1;1]
    lambda, C = eigen(A)
    Ak = C * diag(lambda.^(k-2)) * C^(-1)
    res = Ak * b
    return res[1]
end
```

The **runtime is constant** (for numbers that fall within the range of the IEEE754 floating point representation)

```
@time fib(100)    # 0.000002 seconds
@time fib(1000)   # 0.000002 seconds
```

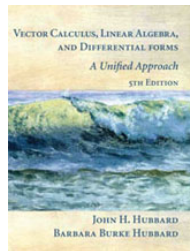
Some references if you have to refresh even more...

ThreeBlueOneBrown on Youtube



https://www.youtube.com/playlist?list=PLZHQObOWTQDPD3MizzM2xVFitgF8hE_ab

The Book by Hubbard and Hubbard



John H Hubbard and Barbara Burke Hubbard. *Vector Calculus, Linear Algebra, and Differential Forms: A Unified Approach*. Matrix Editions, 2015

...or your old course material on LA for that matter!

Newton's Method

Gradient Descent (Recap)

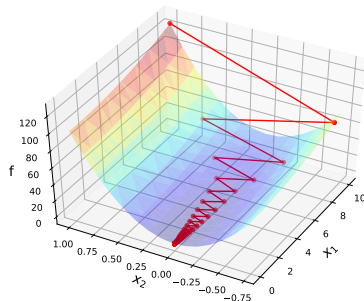
- **Iterative method** starting at an initial point $\mathbf{x}^{(0)}$
- Step to the next point $\mathbf{x}^{(k+1)}$ in the **direction of the negative gradient**

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \nabla f(\mathbf{x}^{(k)})$$

- Repeat until $\|\nabla f(\mathbf{x}^{(k)})\| < \epsilon$ for a chosen ϵ
- **But:** No convergence is guaranteed.
For convergence, an additional **line search** is required.

Line Search

- Take the descent step direction $\mathbf{d} = -\nabla f(\mathbf{x})$
- Select the step length α as $\min_{\alpha \geq 0} f(\mathbf{x} + \alpha \mathbf{d})$
- In practice, α is selected with heuristics



Gradient Descent for

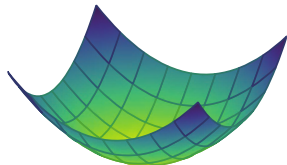
$$f(\mathbf{x}) = \frac{1}{2}(x_1)^2 + 5(x_2)^2$$

Points with Zero Gradient

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$. If f is strictly convex, then it has a single global minimum.
If it is strictly concave, then it has a single global maximum.

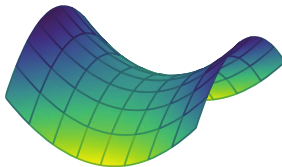
Assume f is **differentiable everywhere**. The extrema are at points x^* with zero gradient $\nabla f(x^*) = \mathbf{0}$.

Strictly Convex



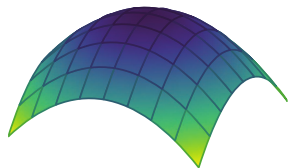
Global Minimum

Neither Concave nor Convex



Possibly a **Saddle-Point** or a
Local Minimum / Maximum

Strictly Concave



Global Maximum

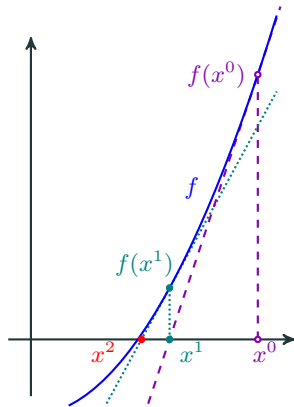
This only works when f is **unconstrained** and defined for $\mathbb{R}^n$. Otherwise the optimum can be “on the edge” of f ’s domain. There, the gradient is not necessarily zero.

Newton's Method for Root-Finding

- Method for finding an x with $f(x) = 0$
- Start with some initial x^0
Iterative improvements: $x^{k+1} = x^k - \frac{f(x^k)}{\nabla f(x^k)}$
- The motivation for the update is to take the subgradient at x^k and find its intersection with the abscissa
- Careful! The Newton Method can diverge.

Search y where the linearization at x intersects with the abscissa.

$$\begin{aligned} 0 &\stackrel{!}{=} f(x) + \nabla f(x)(y - x) \\ \nabla f(x)y &= \nabla f(x)x - f(x) \\ y &= x - \frac{f(x)}{\nabla f(x)} \end{aligned}$$



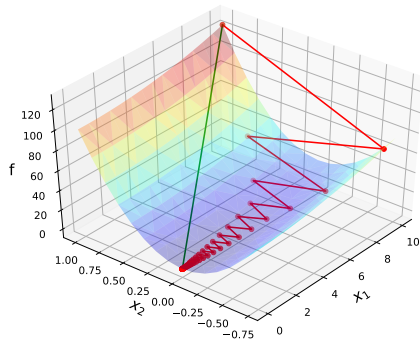
Find $\sqrt{2}$ as the root of $f(x) = x^2 - 2$

Newton's Method for Convex Minimization

- We want to find $\mathbf{x}^*$ with $\nabla f(\mathbf{x}^*) = \mathbf{0}$
- Use **Newton's Method on the derivative of f**

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \underbrace{\left[\nabla^2 f(\mathbf{x}^{(k)}) \right]^{-1}}_{\mathbf{d}} \nabla f(\mathbf{x}^{(k)})$$

- Again, additional Line Search along the step direction $\mathbf{d}$ ensures convergence
- In general, Newton's Method requires **much fewer iterations than Gradient Descent**
- **But:** Requires the **inverse of the Hessian**
 - Computational complexity ca. $O(n^3)$
 - Doesn't scale!



Newton's Method vs. Gradient Descent

Newton's Method as Iterative Taylor Series

- Newton's method is equivalent to the **iterative minimization** of the **2nd degree Taylor Series** of f .
- Take the 2nd degree Taylor Series of f around some point z :
- Find the optimizer for the 2nd degree Taylor Series:

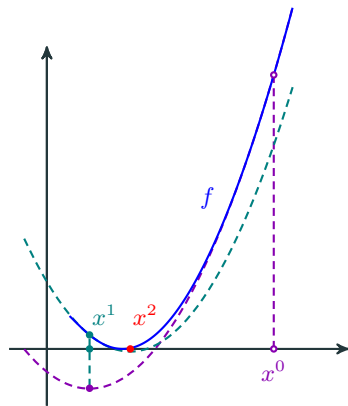
$$\tau_z(x) = f(z) + \underbrace{\nabla f(z)^\top (x - z)}_{\text{1st degree}} + \underbrace{\frac{1}{2}(x - z)^\top [\nabla^2 f(z)] (x - z)}_{\text{2nd degree}}$$

For simplicity of the notation we use $\nabla^2 f(z) = \mathbf{H}$.

$$\nabla \tau_z(x) = \nabla f(z) + \mathbf{H}(x - z) \stackrel{!}{=} \mathbf{0}$$

$$\mathbf{H}x = \mathbf{H}z - \nabla f(z)$$

$$x = z - \mathbf{H}^{-1} \nabla f(z)$$



Super-Convergence of the Newton Method

Theorem: Super-Convergence of Newton's Method [Kantorovich48]

Let f a twice differentiable function with domain $\mathbb{R}^n$. If

1. the Hessian $\nabla^2 f$ is positive definite and Lipschitz for some $l > 0$

$$\|\nabla^2 f(\mathbf{x}) - \nabla^2 f(\mathbf{y})\| < l \|\mathbf{x} - \mathbf{y}\|,$$

2. the initial guess $\mathbf{x}^{(0)}$ is close to the optimum: $\|\mathbf{x}^{(0)} - \mathbf{x}^*\| \leq \frac{1}{2l}$,

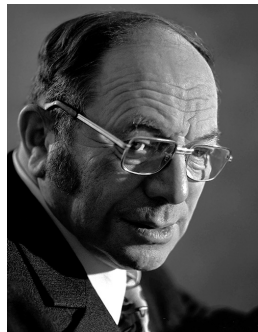
then the Newton-Method converges to $\mathbf{x}^*$ with a **runtime in the order of $O(\log \log \frac{1}{\epsilon})$** for the error ϵ .

This is a **very good speed of convergence!**

$$\log \log 1\,000\,000 \approx 0.778$$

$$\log \log 1\,000\,000\,000 \approx 0.954$$

But: Requires an **initial solution close enough to the minimizer**



Leonid Kantorovich (1912 – 1986)

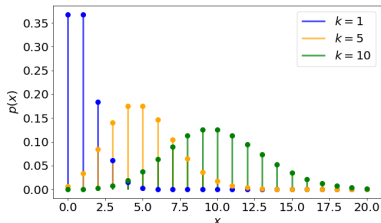
Model Fitting as Optimization

Families of Probability Distributions

Poisson Distribution

$$p_{\theta}(x) = \frac{k^x}{x!} e^{-k}$$

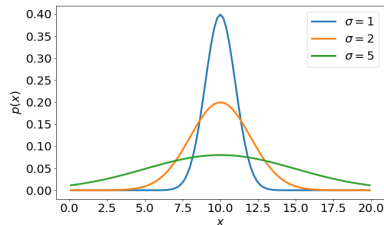
- **Discrete distribution** parameterized by $\theta = (k)$
- If the **interarrival time** of independent events, e.g. of customers in a shop, is described by the **exponential distribution** $p(x) = k e^{-kx}$, then the number of total arrivals per interval of time is Poisson distributed with the same k



Normal Distribution (Gaussian)

$$p_{\theta}(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

- **Continuous Distribution** parameterized by $\theta = (\mu, \sigma)$ for mean and variance
- The sum of many random variables is normally distributed (under some mild assumptions, cf. the **Central Limit Theorem**)



The Likelihood Function

Definition

The likelihood function $\ell_{\theta}(D) = \prod_{i=1}^{|D|} p_{\theta}(x_i)$ indicates how well a dataset $D = \{x_1, x_2, \dots\}$ is described by a probability density function $p_{\theta}(x)$ with parameters θ .

This assumes that all samples x_i from the dataset D are **iid (independent and identically distributed)**. That is often not true in practice.

The likelihood is **not itself a probability distribution**. There are cases where the integral of the likelihood over possible parameters $\int \ell_{\theta}(D) d\theta$ is not one.

Maximizing the likelihood over θ gives the **parameters for the probability distribution that correspond best with the observation data**. Using the negative likelihood turns it into a minimization problem

But: The likelihood can take on very small values which are computationally difficult to handle.

⇒ Minimize the **Negative Log Likelihood Function** instead.

⇒ Logarithm is monotonically increasing, therefore preserves the location of the global minimum.

$$-\log(\ell_{\theta}(D)) = \boxed{-\sum_i^{|D|} \log(p_{\theta}(x_i))}$$

Carbon Dating

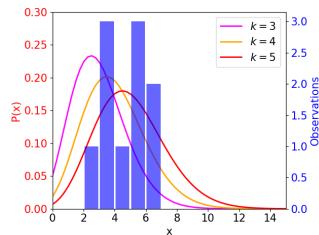
The **Carbon Dating Method** is used to estimate the age of archeological artefacts.

The content of $^{14}_6\text{C}$ in the material reduces over time due to radioactive decay.



The number of **carbon $^{14}_6\text{C}$ decays within a fixed time period** are measured. The number of decays follows a **Poisson distribution**.

The parameter k of the Poisson distribution is estimated from several decay count measurements.



Histogram of number of observed $^{14}_6\text{C}$ decays and different potential Poisson distributions

Carbon Dating 2

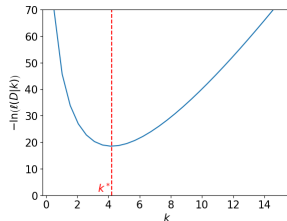
Negative Log Likelihood

$$\begin{aligned} -\log \ell_k(D) &= -\sum_{i=1}^{|D|} \log \left(\frac{k^{x_i}}{x_i!} e^{-k} \right) \\ &= -\sum_{i=1}^{|D|} \left(\log \left(\frac{k^{x_i}}{x_i!} \right) - k \right) \end{aligned}$$

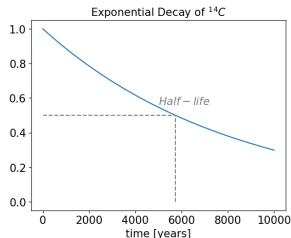
Optimization Problem

$$\min_{k \in \mathbb{R}} \left[|D|k - \sum_{i=1}^{|D|} \log \left(\frac{k^{x_i}}{x_i!} \right) \right]$$

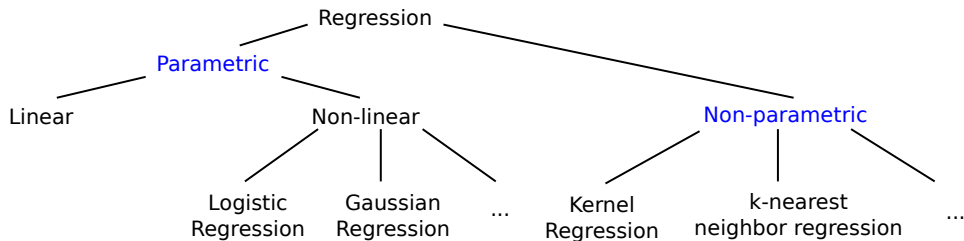
From the optimal solution k^* , the likely age of the archeological artefact can be inferred.



Negative Log Likelihood Function vs. the Poisson distribution parameter $\theta = (k)$



Overview on Regression Types/Model Classes



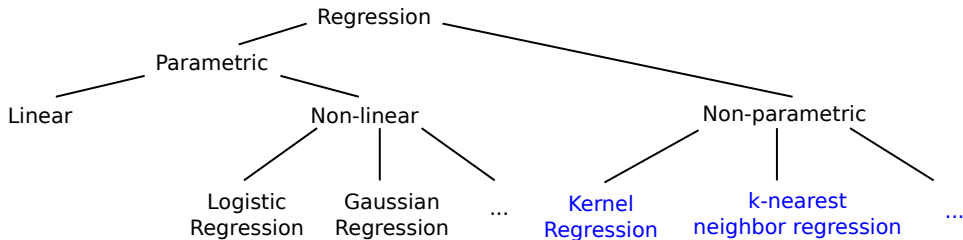
Parametric Regression

- Models assume a finite set of parameters
- Complexity of the model is bounded even if the amount of data is unbounded $\Rightarrow$ Constrained in flexibility

Non-Parametric Regression

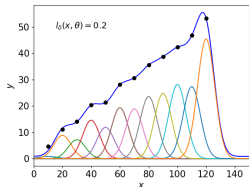
- Models allow for an infinitely large set of parameters
- Amount of information that can be captured grows with the amount of data $\Rightarrow$ more flexible but higher computing and storage requirements

Overview on Regression Types/Model Classes



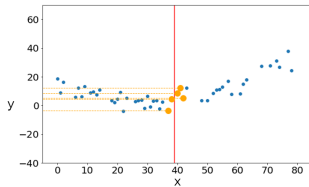
Kernel Regression

Kernel Functions centered at each datapoint $\Rightarrow$
Weighted combination of nearby data points

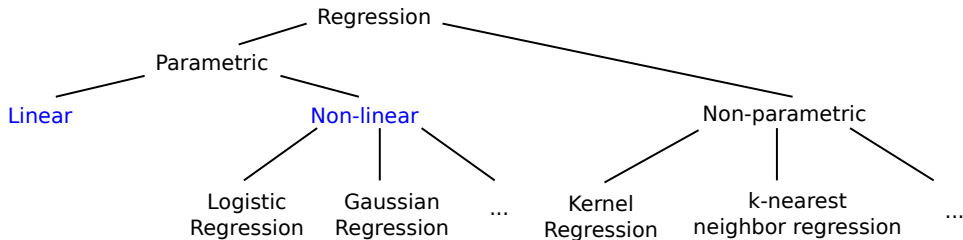


k-Nearest Neighbor Regression

Model at any given point is calculated as the average of the k nearest neighbors



Overview on Regression Types/Model Classes



Linear regression

- **Model output linear** with respect to the model parameters θ
- Examples: Least Squares for fitting of a polynomial
- Closed-form expression to solve for best-fitting parameters available in many cases (**Lecture 7**)

Non-linear regression

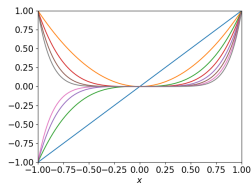
- **Model output non-linear** with respect to the parameters θ
- Example: Neural Network with non-linear activation function
- In general, no closed-form expression available $\Rightarrow$ numerical optimization
- Possibly **local minima** of the loss function

Linear Regression

$$m_{\theta}(x) = \sum_j \theta_j \phi_j(x) = \theta^{\top} \phi(x)$$

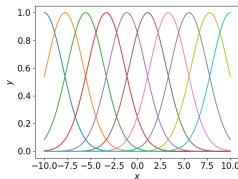
- Model $m_{\theta}(x)$ to be trained on labeled data $D = \{(x_1, y_1), (x_2, y_2), \dots\}$ so that $y_i \approx m_{\theta}(x_i)$
- Finite set of n **Basis Functions** $\phi_j(x)$
(Can be non-linear w.r.t. data x)
- Parameters $\theta = (\theta_1, \theta_2, \dots, \theta_n)$ contain **weights for every basis function**

Polynomial



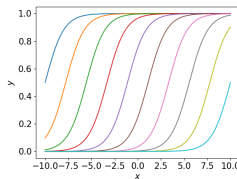
$$\phi_j(x) = x^j$$

Gaussian



$$\phi_j(x) = e^{-\frac{(x-\mu_j)^2}{2\sigma^2}}$$

Sigmoidal



$$\phi_j(x) = \frac{1}{1 + \exp(-\frac{x-\mu_j}{s})}$$

Recap from Lecture 1: Empirical Risk Minimization

- **Given a Dataset** with input-output relations:

$$D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots\}$$

- **Assume a Model Class**, e.g. a polynomial of degree 2:

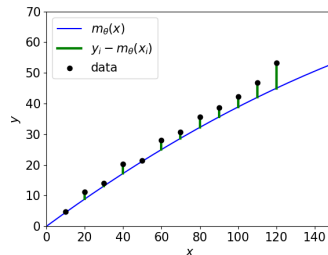
$$m_{\theta}(\mathbf{x}) = \theta_1 + \theta_2 x_1 + \theta_3 x_2^2$$

- **Select a Loss Function**, for examples the average quadratic prediction error (mean squared error or MSE):

$$l_{\theta}(D) = \frac{1}{|D|} \sum_{i=0}^{|D|} (y_i - m_{\theta}(\mathbf{x}_i))^2$$

- **Minimize l** to find the “best” model parameters:

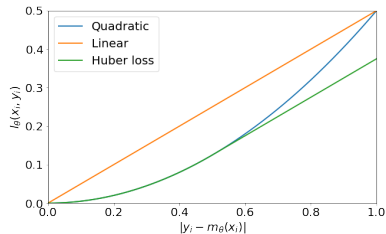
$$\theta^* = \arg \min_{\theta} l(\theta)$$



$$l_{\theta}(D) = \frac{1}{|D|} \sum_{i=1}^{|D|} |y_i - m_{\theta}(\mathbf{x}_i)|$$

Mean Absolute Error (MAE)

- Needs more complicated tools such as linear programming to compute the gradients
- More robust to outliers than quadratic loss since deviation is not squared

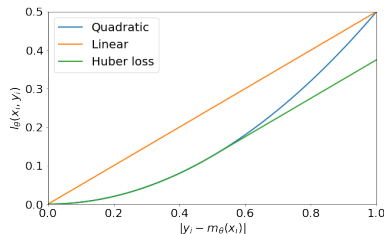


$$l_{\theta}(\omega; D) = \frac{1}{|D|} \sum_{i=1}^{|D|} \begin{cases} \frac{1}{2} \Delta_i^2 & \text{if } \Delta_i \leq \omega \\ \omega \cdot (\Delta_i - \frac{1}{2} \omega), & \text{else} \end{cases}$$

where $\Delta_i = |y_i - m_{\theta}(\mathbf{x}_i)|$

Huber Loss

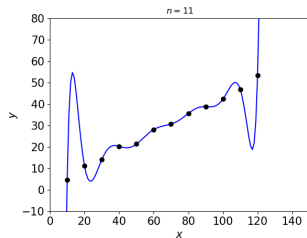
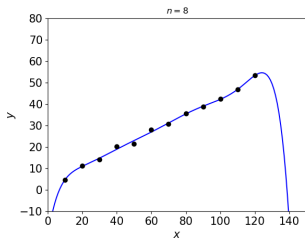
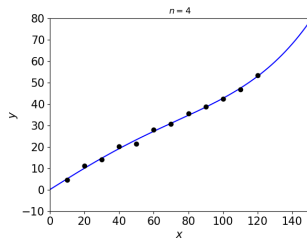
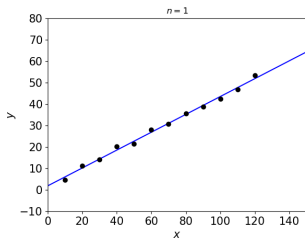
- Combines robustness against outliers of MAE and sensitivity strong convexity around the minimum
- Has one additional parameter ω that has to be tuned



Which is the best model?

Polynomial model + mean squared error minimization:

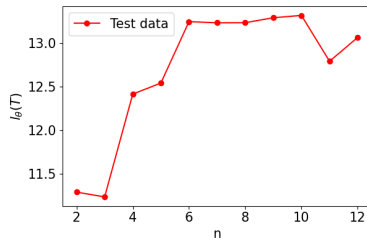
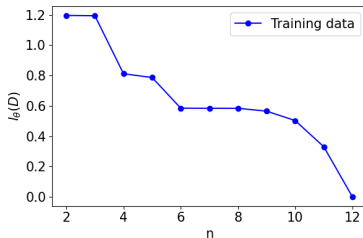
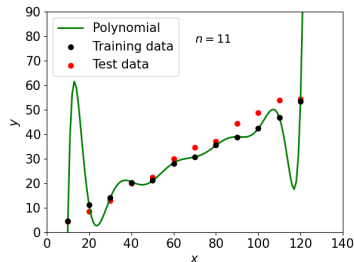
- MSE decreases with increasing model complexity (degree of polynomials n)
- For large n all observed points are predicted perfectly. But the prediction is **too strongly tuned to the data**.



Overfitting

- Split available data into **Training Data D** and **Test Data T**
- For an increasing number of model parameters n :
 - $l_{\theta}(D)$ decreases for the training set
 - $l_{\theta}(T)$ can start to increase for the test set

	Loss	
	Training Data	Test Data
$n = 2$	1.2	11.2
$n = 11$	0.03	13.0



	$n = 2$	$n = 5$	$n = 11$
θ_0	1.8	-5.3	-4300
θ_1	24	79	74000
θ_2	-0.2	-133	-530000
θ_3		148	2100000
θ_4		-76	-5100000
θ_5		14	82000000
θ_6			-9.0
θ_7			6700000
θ_8			-3400000
θ_9			1100000
θ_{10}			-200000
θ_{11}			17000

- Size of each model parameter θ_i tends to grow with an increasing number of parameters n
- Penalize the size of each θ_i while still allowing for a large number of parameters
- Used to prevent overfitting for models with many tunable parameters

Regularization: Penalty term in loss function

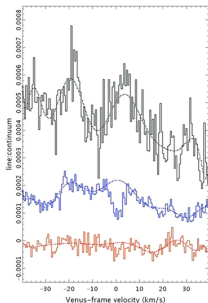
$$\text{minimize} \quad \frac{1}{|D|} \sum_{i=1}^{|D|} (y_i - \boldsymbol{\theta}^\top \boldsymbol{\phi}(x_i))^2 + \mu \sum_{j=1}^n |\theta_j|^q$$

with fixed values $\mu > 0, q \geq 1$

Also with the added penalty term for regularization, minimizing the MSE loss **remains a convex optimization problem!**

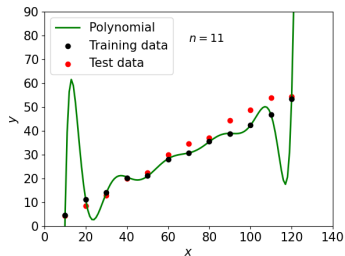
A Current Example for Overfitting

Life on Venus! But is it true?



PH3 spectra illustrating the **12th-order polynomial functions selected empirically** for fitting the spectral ripple seen with ALMA. [Greaves2020]

[Snellen2020] finds serious problems with the original analysis – that had been published in *Nature*!



Fitting a 12-th order polynomial to find small effects in comparatively noisy data is ... wild. Without taking precautions you are likely to be chasing phantoms.

See also this quote from John von Neumann:
With four parameters I can fit an elephant, and with five I can make him wiggle his trunk. [Dyson2004]

Summary of what you learned today

- Repetition of some basic concepts from **Linear Algebra** that will be important going forward
- How the **Newton Method** leads to dramatic speed improvements over Gradient Descent
- How a **Probability Distribution** can be fitted to observations via the maximization of the **Likelihood Function**
- How training of **Regression Models** is turned into an optimization problem via the definition of an appropriate **Loss Function**
- How **Overfitting** can be prevented by an additional **Regularization**

That's it for today.

See you next week for Lecture 3 on
Inequality Constraints

- [Dyson2004] Freeman Dyson. “A meeting with Enrico Fermi”. In: *Nature* 427 (2004), pp. 297–297.
- [Greaves2020] Jane S Greaves et al. “Phosphine gas in the cloud decks of Venus”. In: *Nature Astronomy* (2020), pp. 1–10.
- [Hubbard2015] John H Hubbard and Barbara Burke Hubbard. *Vector Calculus, Linear Algebra, and Differential Forms: A Unified Approach*. Matrix Editions, 2015.
- [Kantorovich48] Leonid Vital’evich Kantorovich. “Functional analysis and applied mathematics”. In: *Uspekhi Matematicheskikh Nauk* 3.6 (1948), pp. 89–185.
- [Snellen2020] IAG Snellen et al. “Re-analysis of the 267-GHz ALMA observations of Venus: No statistically significant detection of phosphine”. In: *arXiv preprint arXiv:2010.09761* (2020).